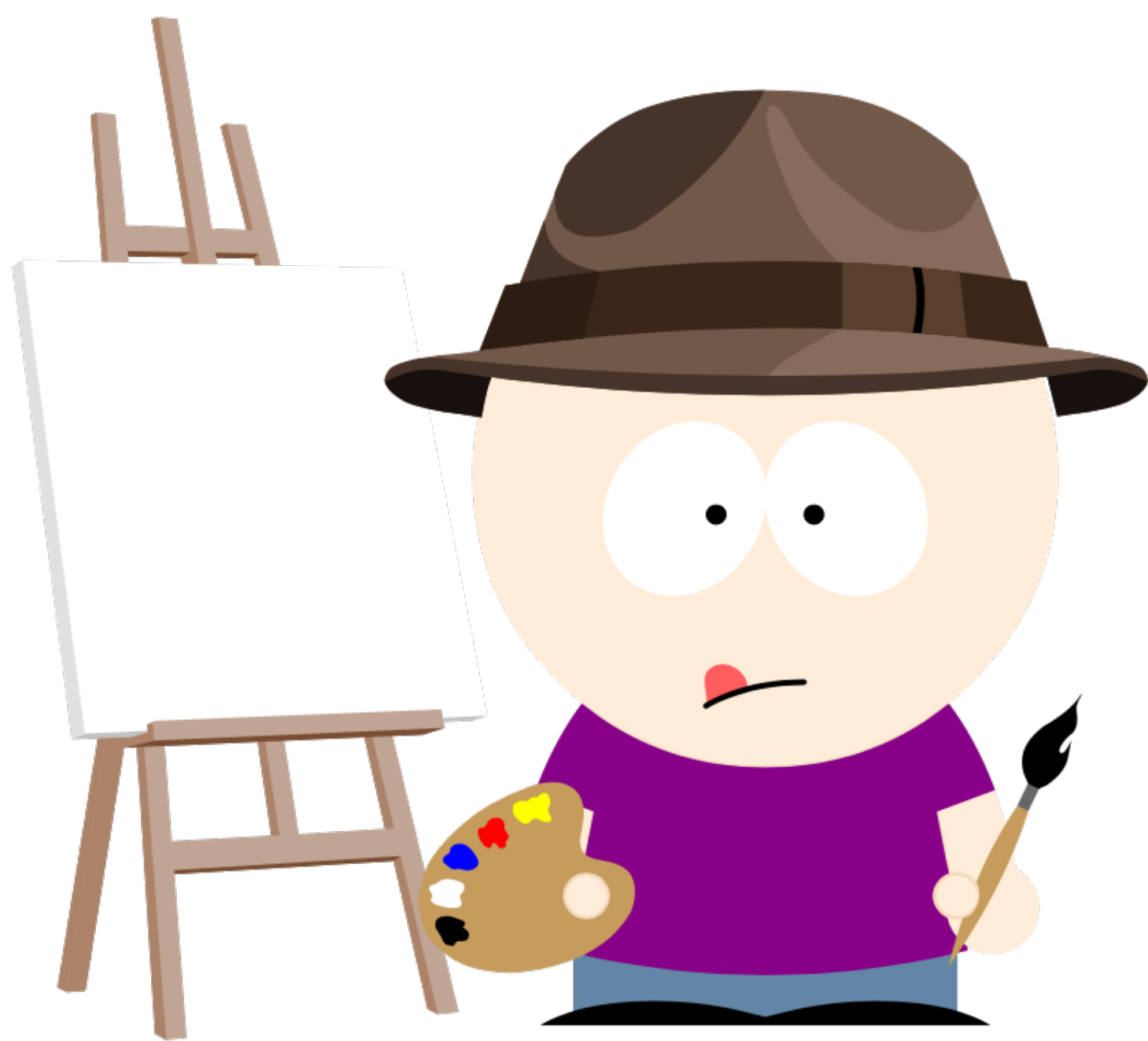




IN-CLASS EXERCISES



CODE OPTIMIZATION 2

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`opt-2-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip opt-2-exercises.zip
```

```
> cd opt-2-exercises
```

```
> ls
```

Task 1

- Describe these optimizing transformations:
 1. Dead code elimination
 2. Common subexpression elimination
 3. Loop unrolling
 4. Code hoisting
 5. Induction variable removal
 6. Reduction in strength
 7. Copy propagation
 8. Inline expansion
 9. Procedure cloning
 10. Inter-procedural constant propagation

Task 2

1. Apply an *inline expansion* transformation to the program in `task2.c`. Make sure that the program works the same after the transformation.
2. Based on your experience, design an inline expansion algorithm.

```
void P (int x, int* y) {  
    x = *y + 2;  
    *y = *y - 1;  
    int i = 0;  
    while (i < x) {  
        printf("Hi!\n");  
        i++;  
    };  
}
```

```
int main() {  
    int a = 3;  
    int b = 5;  
    P(a, & b);  
    int i = 0;  
    while (i < (a+b)) {  
        printf("By!\n");  
        i++;  
    }  
}
```

Task 3

- Apply the *Common Subexpression Elimination* transformation to the program in `task3.c` (also shown the next page).
- Find as many subexpressions as possible to eliminate without changing the behavior of the program.

```
int main() {
    int a = 2;
    int b = 3;
    int i = 0;
    while (i < (a+b)) {
        if ((i%2)==0) {
            printf("A %i\n", i);
        };
        i++;
        if ((i%2)==0) {
            printf("B %i\n", i);
        };
        int x = (i < (a+b)) + 1;
        for(int j=0; j<x; j++){
            printf("C %i\n", i%2);
        }
    }
}
```


Task 4

- Apply the *Code Hoisting* transformation to the program in `task4.c` (also shown the next page).
- Find as many subexpressions as possible to hoist as far up as possible to make the program as fast as possible.

```
int main(int argc, char** argv) {
    int a = argc + 2;
    int A[] = {
        0,1,2,3,4,5,6,7,8,9,10,11,12,13,
        14,15,16,17,18,19,20};
    int i = 0;
    while (i < 8) {
        int x;
        if ((i%2)==0) {
            x = A[i] * A[2*i+1];
        } else {
            x = A[2*i+1] + 4;
            x = x + (a * 9);
        };
        printf("A %i\n", x);
        i++;
    }
}
```


Task 5

- Apply the *Loop Unrolling* transformation to the loops in functions P and Q in task5.c (also shown here and on the next page).
- You only have to unroll each loop *once*.

```
int main(int argc, char** argv) {  
    P();  
    int A[] = {0,1,2,3,4,5,6,7,8,...};  
    int B[] = {0,1,2,3,4,5,6,7,8,...};  
    Q(A,B,10);  
    Q(A,B,15);  
}
```

```
void P() {  
    int A[] = {0,1,2,3,4,5,6,7,8,...};  
    int B[] = {0,1,2,3,4,5,6,7,8,...};  
    int s = 0;  
    for(int i=0; i<20; i++) {  
        s += A[i] + B[i];  
    };  
    printf("A %i\n", s);  
}
```

```
void Q(int A[], int B[], int n) {  
    int s = 0;  
    for(int i=0; i<n; i++) {  
        s += A[i] + B[i];  
    };  
    printf("B %i\n", s);  
}
```




DEFINITIONS AND ALGORITHMS

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA